

Patrón Decorator

1. Concepto y categoría a la que pertenece (Creational, Structural, Behavioral)

El patrón de diseño Decorador, propuesto en 1994, permite extender las funcionalidades de una clase de forma dinámica en la programación orientada a objetos. Este enfoque hace posible añadir nuevos comportamientos a un objeto específico sin alterar la funcionalidad de otros objetos de la misma clase. Aunque estructuralmente guarda ciertas similitudes con el patrón Cadena de Responsabilidad, donde las clases gestionan una secuencia de peticiones, el Decorador se diferencia porque no depende de un único procesador central.

La idea detrás de este patrón es "decorar" el componente de software que deseamos ampliar, envolviéndolo con una o más clases decoradoras que tienen la misma interfaz. Esto asegura que las llamadas de métodos al objeto puedan ser delegadas o procesadas dentro del decorador, añadiendo las nuevas funcionalidades sin modificar la estructura interna del componente original.

Categoría: Structural (Estructural)

2. Con base en el concepto, que problema resuelve o cual es el escenario de utilización.

El patrón Decorator es útil cuando quieres agregar responsabilidades o funcionalidades a objetos individuales, sin modificar la clase original.

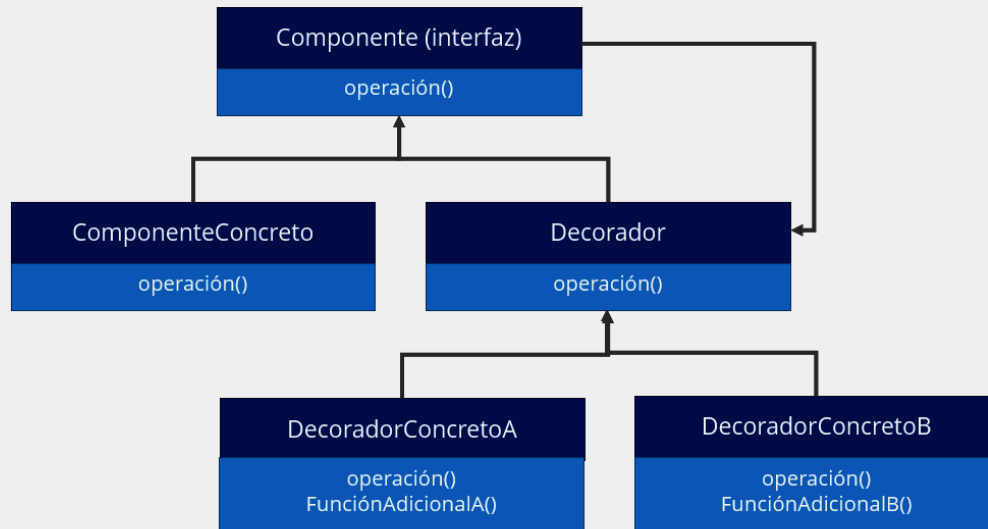
Escenarios de uso:

Cuando necesitas añadir funcionalidades a objetos individuales y no a todas las instancias de una clase.

- Cuando deseas evitar el uso de herencia para agregar comportamientos adicionales.
- Cuando los comportamientos adicionales pueden combinarse de forma dinámica en tiempo de ejecución.

3. Diagrama de clase y Diagrama de secuencia del patrón de diseño

Diagrama UML: patrón de diseño Decorator



IONOS

4. Ejemplo en código fuente de su utilización.

```
Main.java > Main > main(String[])
1  // 1. Componente base
2  interface Notificador {
3      void enviar(String mensaje);
4  }
5
6  // 2. Componente concreto
7  class NotificadorBasico implements Notificador {
8      @Override
9      public void enviar(String mensaje) {
10         System.out.println("Enviando notificación: " + mensaje);
11     }
12 }
13
14 // 3. Decorador abstracto
15 abstract class NotificadorDecorator implements Notificador {
16     protected Notificador notificador;
17
18     public NotificadorDecorator(Notificador notificador) {
19         this.notificador = notificador;
20     }
21
22     @Override
23     public void enviar(String mensaje) {
24         notificador.enviar(mensaje);
25     }
26 }
```

Main.java > Main > main(String[])

```
28 // 4. Decoradores concretos
29 class NotificadorEmail extends NotificadorDecorator {
30     public NotificadorEmail(Notificador notificador) {
31         super(notificador);
32     }
33
34     @Override
35     public void enviar(String mensaje) {
36         super.enviar(mensaje);
37         enviarEmail(mensaje);
38     }
39
40     private void enviarEmail(String mensaje) {
41         System.out.println("Enviando notificación por Email: " + mensaje);
42     }
43 }
44
45 class NotificadorSMS extends NotificadorDecorator {
46     public NotificadorSMS(Notificador notificador) {
47         super(notificador);
48     }
49
50     @Override
51     public void enviar(String mensaje) {
52         super.enviar(mensaje);
53         enviarSMS(mensaje);
54     }
55
56     private void enviarSMS(String mensaje) {
57         System.out.println("Enviando notificación por SMS: " + mensaje);
58     }
59 }
60
61 // Uso del patrón
62 public class Main {
63     public static void main(String[] args) {
64         Notificador notificadorBasico = new NotificadorBasico();
65
66         // Añadir comportamiento adicional (Decorator)
67         Notificador notificadorConEmail = new NotificadorEmail(notificadorBasico);
68         Notificador notificadorConSMS = new NotificadorSMS(notificadorConEmail);
69
70         // Llamada con todos los decoradores aplicados
71         notificadorConSMS.enviar("Hola, usuario!");
72     }
73 }
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
Enviando notificación: Hola, usuario!
Enviando notificación por Email: Hola, usuario!
Enviando notificación por SMS: Hola, usuario!
PS C:\Users\Elvis Crespo\Downloads\registro_seminarios\registro_seminarios>
```

Bibliografía

Decorator. (s/f). Refactoring.guru. Recuperado el 2 de octubre de 2024, de <https://refactoring.guru/es/design-patterns/decorator>

Patrón Decorator: ampliación dinámica de clases. (2021, febrero 19). IONOS Digital Guide; IONOS. <https://www.ionos.com/es-us/digitalguide/paginas-web/desarrollo-web/que-es-el-patron-de-diseno-decorator/>